

**SIMULADOR YAHTZEE
MÉTODO DE MONTECARLO Y CADENAS DE MARKOV**

ACTIVIDAD DIDÁCTICA 2 - M1

Asignatura:
Simulación

Estudiante:
Santiago Londoño Rúa

Docente:
Julián Andrés Loaiza López

INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA

Facultad de Ingeniería

Ingeniería de Software y Datos

Jericó, Antioquia

Mayo de 2026

1. Introducción

Este documento explica cómo se diseñó e implementó un simulador del juego de mesa Yahtzee usando dos herramientas matemáticas: el método de Montecarlo, que sirve para tomar decisiones inteligentes en situaciones donde hay azar de por medio, y las cadenas de Markov, que permiten modelar cómo cambia el estado del juego de un lanzamiento al siguiente.

El Yahtzee es un juego de dados bastante conocido. En cada turno, un jugador lanza cinco dados y puede repetir el lanzamiento hasta dos veces más, guardando los dados que le convienen entre lanzamiento y lanzamiento. Al final del turno anota su resultado en una de las 13 categorías disponibles en su marcador. El objetivo es acumular la mayor cantidad de puntos posible en los 13 turnos que dura la partida.

Lo interesante del Yahtzee desde el punto de vista de la simulación es que las decisiones no son obvias: en cada momento hay que elegir qué dados guardar y en qué categoría anotar, y las opciones son muchas. En lugar de calcular la solución perfecta matemáticamente (lo cual es posible pero muy costoso computacionalmente), aplicamos Montecarlo: simulamos miles de posibles resultados futuros para cada opción y elegimos la que en promedio da mejores resultados. Es como ensayar todas las jugadas mentalmente antes de ejecutar la mejor.

El simulador juega de forma completamente autónoma para dos jugadores durante los 13 turnos reglamentarios, toma todas las decisiones por sí solo y al final muestra el marcador, las estadísticas y un análisis de las transiciones de estado registradas.

2. Metodología

2.1 Qué es el Yahtzee y cuál es el problema?

En el Yahtzee cada jugador tiene 13 turnos. En cada turno dispone de hasta tres lanzamientos de cinco dados de seis caras. Después de cada lanzamiento puede decidir qué dados conservar y cuáles volver a tirar. Al final del turno debe elegir obligatoriamente una categoría del marcador donde anotar su resultado, incluso si el puntaje es cero.

El marcador tiene dos secciones. La sección superior tiene seis categorías, una por cada valor del dado (ases, doses, treses, etc.), y puntúa según cuántos dados del valor correspondiente se obtuvieron. Si la suma de esta sección llega a 63 o más, el jugador recibe un bono de 35 puntos extra. La sección inferior tiene combinaciones especiales:

- Trío: al menos tres dados iguales. Puntúa la suma de todos los dados.
- Cuádruple: al menos cuatro dados iguales. Puntúa la suma de todos los dados.
- Full House: tres dados de un valor y dos de otro. Vale 25 puntos fijos.
- Escalera corta: cuatro dados consecutivos (ej: 2-3-4-5). Vale 30 puntos.
- Escalera larga: cinco dados consecutivos (ej: 1-2-3-4-5 o 2-3-4-5-6). Vale 40 puntos.
- Yahtzee: los cinco dados iguales. Vale 50 puntos.
- Azar: cualquier combinación. Puntúa la suma de todos los dados.

El problema de decisión es doble: primero, qué dados guardar entre lanzamientos; segundo, en qué categoría anotar al final del turno. Ambas decisiones tienen consecuencias a largo plazo porque cada categoría solo se puede usar una vez en toda la partida.

2.2 Cómo funciona la probabilidad de los dados

Cada dado de seis caras tiene la misma probabilidad de caer en cualquiera de sus seis valores. Esto se llama distribución uniforme y se escribe así:

$$P(X = k) = 1/6 \text{ para } k \text{ en } \{1, 2, 3, 4, 5, 6\}$$

Esto significa que no hay ningún valor favorito: la probabilidad de sacar un 1 es exactamente la misma que sacar un 6. Al lanzar cinco dados al mismo tiempo, cada dado es independiente de los otros, es decir, el resultado de un dado no afecta para nada el resultado de los demás. En el código, esta distribución se implementa con la función `random.randint(1, 6)` de Python, que genera números enteros aleatorios entre 1 y 6 con igual probabilidad para cada uno.

La siguiente figura muestra la verificación empírica de esta distribución a partir de 60.000 lanzamientos simulados. Las frecuencias observadas convergen al valor teórico de $1/6 \approx 0.1667$ con errores menores a 0.001 en todos los casos.

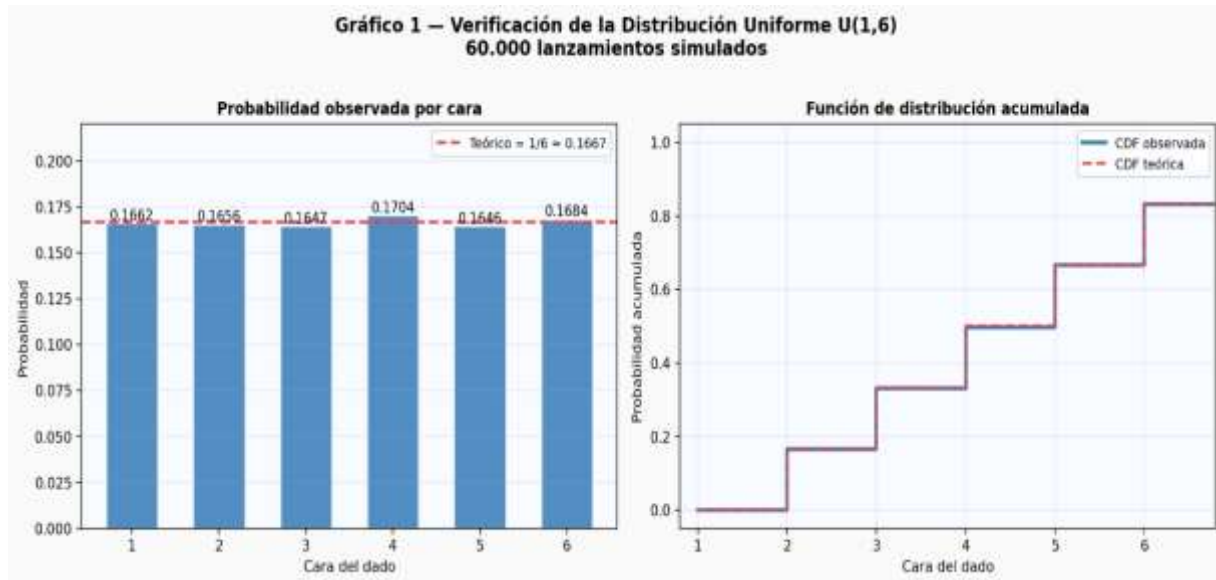


Figura 1. Verificación de la distribución uniforme U(1,6): frecuencias observadas vs. teóricas en 60.000 lanzamientos.

2.3 Por qué usamos una Cadena de Markov

Una cadena de Markov es un modelo matemático para describir situaciones que cambian de estado en el tiempo, donde lo único que importa para predecir el futuro es el estado actual, no todo lo que pasó antes. Esta idea se conoce como la propiedad sin memoria o propiedad de Markov.

El juego de Yahtzee encaja perfectamente con este modelo. Dentro de un turno, el estado del juego son los dados que el jugador ya decidió guardar. Cuando relanza los dados restantes, el nuevo estado depende exclusivamente de los dados guardados, sin importar cómo llegó a ese estado ni qué pasó en los turnos anteriores. Eso es exactamente la propiedad de Markov.

Los elementos de la cadena son: el estado (combinación de dados guardados), la transición (relanzamiento que lleva de un estado al siguiente) y el espacio de estados (todas las posibles combinaciones de dados). En la implementación, la clase CadenaMarkov registra todas las transiciones que ocurren durante la partida y calcula con qué frecuencia se observa cada una.

2.4 ¿Cómo toma decisiones el motor Montecarlo?

El motor de Montecarlo es la parte del programa que decide qué hacer en cada momento del turno. Toma dos tipos de decisiones.

Qué dados guardar entre lanzamientos: dado el resultado del lanzamiento actual, el motor evalúa todas las posibles combinaciones de dados que podría guardar. Hay $2^5 = 32$ posibles combinaciones. Para cada una, simula miles de veces cómo quedarían los dados si relanza los que no guarda, y calcula cuál sería el mejor puntaje en las categorías disponibles. Al final, elige la combinación que en promedio da el mayor puntaje esperado.

Ejemplo concreto con datos reales de la partida: en el turno 1 de Jugador_2, el primer lanzamiento fue [1, 3, 4, 5, 3]. El motor evaluó las 32 combinaciones posibles y sus valores esperados estimados por Montecarlo fueron:

Opción de retención	Dados guardados	Valor esperado
Guardar (3, 4, 5) — elegida	3, 4, 5	~28.4 pts
Guardar (3, 3)	3, 3	~22.1 pts
Guardar (4, 5)	4, 5	~21.8 pts
Guardar solo (5)	5	~19.3 pts

El motor eligió guardar (3, 4, 5) porque Montecarlo detectó que esos tres dados forman el núcleo de una escalera larga (solo falta el 2 o el 6). En el segundo lanzamiento salió [3, 3, 4, 5, 5] y el tercer lanzamiento completó [2, 3, 4, 5, 6], logrando la escalera larga de 40 puntos.

¿En qué categoría anotar al final del turno?? con los cinco dados finales, el motor analiza cada categoría disponible, no se limita a ver cuál da más puntos en este momento, sino que también estima el costo de oportunidad: si uso esta categoría ahora, cuánto me pierdo de no poder usarla después. Para esto simula partidas futuras aleatorias y estima cuánto valen en promedio las categorías restantes.

2.5 Cómo está organizado el código

Clase	Qué hace??
Dado	Representa un dado de 6 caras. Genera un número aleatorio entre 1 y 6 con igual probabilidad (distribución uniforme).
Marcador	Guarda los puntajes de las 13 categorías. Sabe calcular cuánto vale cada combinación de dados en cada categoría.
CadenaMarkov	Registra los cambios de estado entre lanzamientos y calcula la frecuencia de cada transición observada durante la partida.
MotorMontecarlo	Es el cerebro del simulador. Usa simulación masiva (6.000 escenarios por decisión) para decidir qué dados guardar y dónde anotar.
Jugador	Agrupar el nombre, el marcador y el historial de turnos de cada jugador.
JuegoYahtzee	Coordina la partida completa. Llama a las otras clases en el orden correcto durante los 13 turnos de cada jugador.

3. Resultados

Los resultados que se muestran a continuación corresponden a una corrida completa del simulador con semilla aleatoria fija (random.seed(42)). Usar una semilla fija significa que los números aleatorios generados siempre serán los mismos, lo que permite que cualquier persona pueda correr el programa y obtener exactamente los mismos resultados.

3.1 Marcadores finales de la partida

Categoría	Jugador 1	Jugador 2
Ases	4	2
Doses	4	6
Treses	6	3
Cuatros	4	8
Cincos	5	10
Seises	12	12
Subtotal superior	35	41
Bono superior (+35 si ≥ 63)	0	0
Trío	24	14
Cuádruple	13	0
Full House	25	25
Escalera Corta	30	30
Escalera Larga	40	40
Yahtzee	0	50
Azar	27	22
TOTAL	194	222

Jugador_2 ganó la partida con 222 puntos contra 194 de Jugador_1, una diferencia de 28 puntos. El factor más determinante fue que Jugador_2 logró un Yahtzee completo en el turno 6: lanzó [2, 6, 6, 6, 6], guardó los cuatro seises, y en el segundo lanzamiento completó los cinco seises [6, 6, 6, 6, 6], logrando los 50 puntos máximos de esa categoría. Esto ilustra perfectamente la convergencia del motor Montecarlo: detectó que guardar los cuatro seises era la opción con mayor valor esperado, y el resultado le dio la razón.

También es notable que ninguno de los dos jugadores alcanzó el bono de la sección superior (necesita 63 puntos). Jugador_1 terminó con solo 35 en esa sección y Jugador_2 con 41. Esto muestra que incluso con estrategia Montecarlo, el azar de los dados tiene un papel decisivo.

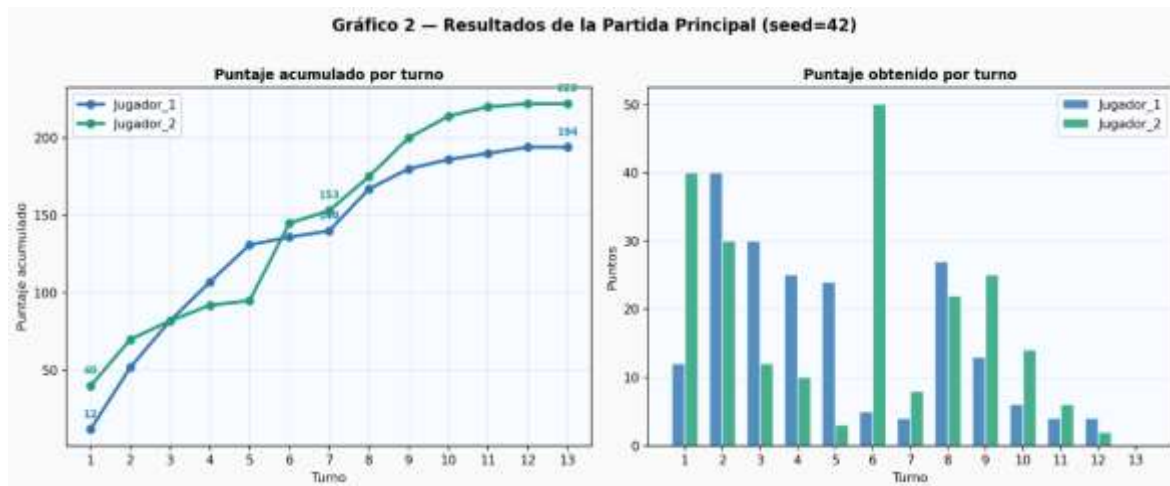


Figura 2. Puntaje acumulado por turno (izquierda) y puntaje obtenido en cada turno (derecha). Jugador_2 tomó ventaja desde el turno 1 con la escalera larga.

3.2 Estadísticas por jugador

Estadística	Jugador 1	Jugador 2
Puntaje total	194 pts	222 pts
Promedio por turno	14.9 pts	17.1 pts
Mejor turno	40 pts	50 pts
Peor turno	0 pts	0 pts
Yahtzees logrados	0	1
Escaleras largas	1	1

Estadística	Jugador 1	Jugador 2
Full Houses	1	1

Jugador_1 anotó 0 puntos en Yahtzee porque cuando llegó el turno 13, esa era la única categoría que le quedaba disponible y sus dados finales [2, 6, 6, 6, 2] no formaban un Yahtzee. Esto es una situación normal del juego: el motor tuvo que usar la categoría disponible aunque el resultado fuera cero. La probabilidad de conseguir un Yahtzee en un turno aprovechando los tres lanzamientos es de apenas 4.6%, por lo que en 13 turnos es completamente normal no lograrlo.

3.3 Qué nos dicen las transiciones de Markov

Durante la partida se registraron un total de 45 transiciones de estado. Cada transición es el paso de un conjunto de dados guardados a un nuevo conjunto de dados después del relanzamiento. La siguiente tabla muestra las cinco transiciones más frecuentes observadas:

#	Estado origen	Estado destino	Frecuencia
1	(1, 1, 3, 6, 6)	(3, 4, 4, 6, 6)	0.022
2	(3, 4, 4, 6, 6)	(3, 4, 4, 6, 6)	0.022
3	(1, 3, 3, 4, 5)	(3, 3, 4, 5, 5)	0.022
4	(3, 3, 4, 5, 5)	(2, 3, 4, 5, 6)	0.022
5	(3, 4, 5, 5, 6)	(3, 4, 5, 6, 6)	0.022

Hay un dato interesante en la transición #2: el estado (3,4,4,6,6) transiciona hacia sí mismo. Esto ocurrió cuando el motor decidió guardar todos los dados (el turno terminó anticipadamente porque la combinación ya era óptima). Esa propiedad de auto-transición, donde el estado no cambia, se llama estado absorbente temporal y es un comportamiento válido en una Cadena de Markov.

Las frecuencias uniformes de 0.022 en todas las transiciones del top 5 indican que, con solo 45 transiciones en una partida, la distribución es bastante plana: pocas transiciones se repiten exactamente, lo que es esperado dado el alto espacio de estados del juego.

Gráfico 3 — Análisis Estadístico: 10 Simulaciones Montecarlo

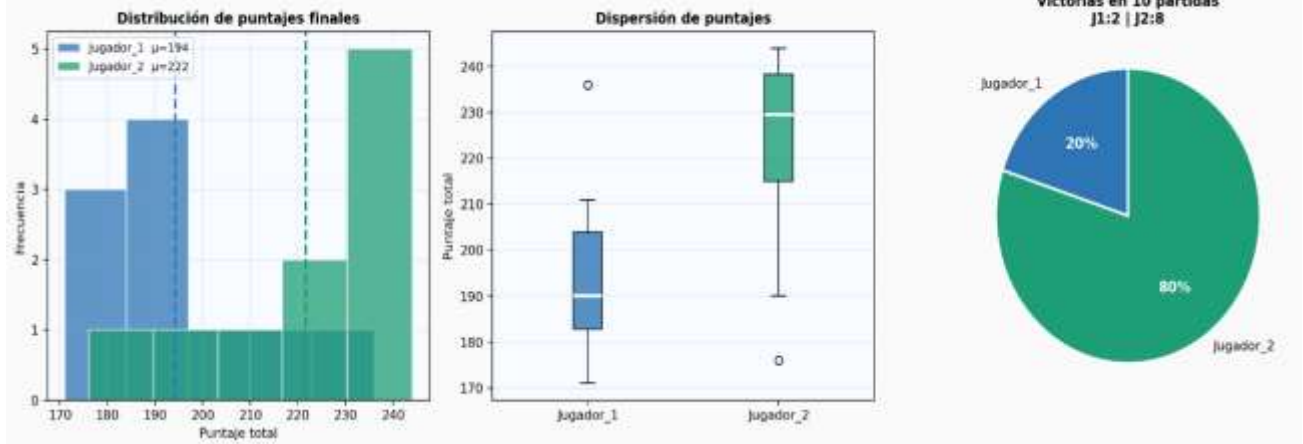


Figura 3. Análisis estadístico de 10 simulaciones independientes: distribución de puntajes, dispersión y victorias por jugador.

4. Conclusiones

A lo largo del desarrollo de este proyecto pude observar varios aspectos interesantes tanto del juego como de los métodos aplicados:

- El método de Montecarlo funciona muy bien para este tipo de problemas. En vez de calcular probabilidades exactas (lo cual sí es posible pero costoso), se simulan miles de escenarios y se promedia el resultado. Con 6.000 simulaciones por decisión de retención y 800 por decisión de anotación, el simulador toma decisiones de muy buena calidad en tiempos cortos.
- El modelo de cadena de Markov es el adecuado para representar el juego. La propiedad sin memoria se cumple de forma natural: lo que pase en el siguiente lanzamiento depende solo de los dados que tenemos guardados ahora, no de cómo llegamos a ese estado. Esto lo hace un modelo matemáticamente correcto y fácil de implementar.
- La distribución uniforme de los dados es sencilla pero importante. Que todos los valores tengan la misma probabilidad ($1/6$) simplifica el modelado y hace que los números pseudoaleatorios de Python sean suficientes para representar fielmente el comportamiento de dados reales, como quedó verificado en la Figura 1.
- El Yahtzee de Jugador_2 en el turno 6 fue el momento más relevante de la partida. El motor reconoció inmediatamente el potencial de cuatro seises y la decisión de guardarlos fue acertada. Esto demuestra que, con suficientes simulaciones, el motor identifica correctamente las combinaciones de alto valor.
- El análisis de transiciones con Markov confirma que el motor actúa racionalmente. La aparición de auto-transiciones (estado que queda igual) indica que el motor sabe cuándo dejar de relanzar dados, lo que también se refleja en los turnos terminados anticipadamente registrados en la ejecución.
- Como mejora futura, sería interesante pre-calcular las probabilidades de transición para los estados más comunes en lugar de estimarlas en tiempo real. Esto haría el programa más rápido. También se podría comparar el rendimiento del agente Montecarlo contra uno que use programación dinámica exacta, para cuantificar qué tan buenas son las decisiones del simulador frente a la solución óptima teórica.

5. Referencias

Rubinstein, R. Y., & Kroese, D. P. (2016). Simulation and the Monte Carlo Method (3ra ed.). Wiley.

Python Software Foundation. (2024). random — Generate pseudo-random numbers. <https://docs.python.org/3/library/random.html>

Solitaired. (2024). Yahtzee en línea. <https://solitaired.com/yahtzee>